# Unsupervised Medical Anomaly Detection via Latent Diffusion Models

## Complete System Workflow & Model Reference

*Soham Patel — Master of Artificial Intelligence*
*OTH Regensburg · Deep Vision · Semester 4*

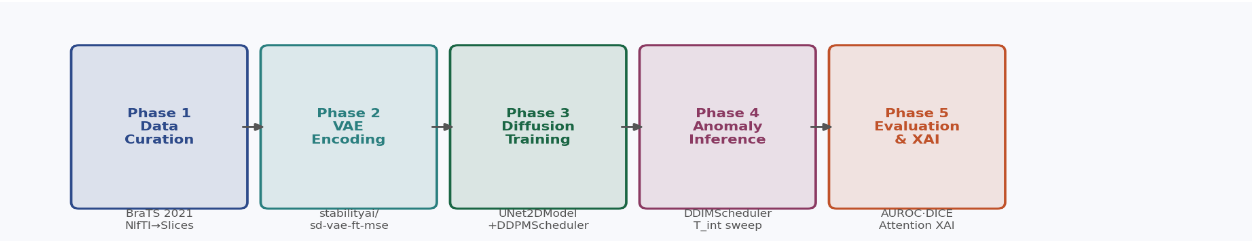


*Figure 1. End-to-end pipeline overview.*

This document specifies the complete technical workflow for unsupervised brain tumour anomaly detection using a Latent Diffusion Model (LDM) trained exclusively on healthy brain MRI slices from the BraTS 2021 dataset. The system learns the normative distribution of healthy brain tissue in a compressed latent space. At inference, anomalous regions are localised by partially noising a test image and measuring the pixel-wise residual between the original and the model's reconstruction of a healthy equivalent. No tumour labels are used during training.

## 1. Model Registry

The following table enumerates every model and scheduler component used in this pipeline, its HuggingFace identifier, the pipeline phase it belongs to, and whether its weights are frozen or trained.

| Component | HuggingFace / Library ID | Phase | Weights | Purpose |
|---|---|---|---|---|
| VAE Encoder/Decoder | `stabilityai/sd-vae-ft-mse` | 1, 2, 4 | Frozen | Compress 256×256×3 → 32×32×4 latents and decode back |
| Diffusion UNet | `diffusers.UNet2DModel (scratch)` | 3 | Trained | Learn healthy brain manifold P(z_healthy) via noise prediction |
| DDPM Scheduler | `diffusers.DDPMScheduler` | 3 | No weights | Define noising process and MSE training objective |
| DDIM Scheduler | `diffusers.DDIMScheduler` | 4 | No weights | Deterministic 50-step reverse denoising for reproducible reconstruction |

| Component | HuggingFace / Library ID | Phase | Weights | Purpose |
|---|---|---|---|---|
| EMA Shadow Model | `torch deepcopy + decay=0.9999` | 3, 4 | Derived | Smooth EMA of UNet weights — used exclusively for inference |

## 2. Phase-by-Phase Technical Specification

| PHASE 1 | Data Curation and Slice Extraction |
|---|---|

**Dataset:** BraTS 2021 (Brain Tumour Segmentation Challenge)

The BraTS 2021 dataset provides 3D multi-modal brain MRI volumes in NIfTI format (.nii.gz) for 1,251 patients, each with four modalities: T1, T1ce (contrast-enhanced), T2, and FLAIR, along with expert-annotated tumour segmentation masks. All volumes have a standardised shape of 240×240×155 voxels at 1mm isotropic resolution.

### Modality Selection: T1ce + T2 + FLAIR (3-channel input)

T1 is discarded. The three retained modalities form the 3-channel input to the VAE: T1ce captures the contrast-enhancing tumour core, T2 captures peritumoral oedema, and FLAIR captures the full lesion region including infiltration. This selection is not arbitrary — it mirrors the clinical reading protocol for glioma diagnosis and provides the VAE with complementary structural signals.

### Processing Steps:

**1.** Load NIfTI volumes via nibabel for all four modalities and the segmentation mask.

**2.** Apply z-score normalisation per modality using only non-zero (brain) voxels. Clip to [-3σ, +3σ] and rescale to [-1, 1] — the input range expected by the SD VAE.

**3.** Extract all 155 axial slices per patient (dimension index 2).

**4.** Pad each 240×240 slice to 256×256 with 8 pixels of zero-padding per side.

**5.** Filter: discard any slice where fewer than 5% of voxels are non-zero across all modalities.

**6.** Segregate: slices where the segmentation mask is entirely zero → training set (healthy). Slices with any non-zero mask voxel → test set (anomalous). Save masks alongside test slices.

**7.** Stack T1ce, T2, FLAIR into a (3, 256, 256) float32 array and save as .npy.

**Expected output:** ~20,000 healthy slices (training) and ~2,000 tumour slices (test).

| PHASE 2 | Latent Space Projection via Frozen VAE |
|---|---|

**Model:** `stabilityai/sd-vae-ft-mse` (KL-regularised Variational Autoencoder, HuggingFace diffusers)

The SD VAE is a convolutional encoder-decoder with a KL-regularised bottleneck. It applies an 8× spatial compression: a 256×256 input is projected to a 32×32×4 latent tensor. The KL penalty enforces a smooth, continuous latent manifold, which is necessary for the diffusion model's Gaussian noise schedule to operate coherently. The VAE is loaded with frozen weights for the entire project — it is never fine-tuned.

$$z = \mathcal{E}(x) \cdot 0.18215, \quad z \in \mathbb{R}^{4 \times 32 \times 32}$$

*Equation 1. VAE encoding with latent scaling factor.*

The scaling factor 0.18215 is the empirically derived constant from the SD VAE training that normalises the latent distribution variance. Omitting it causes the diffusion model's noise schedule to operate on the wrong magnitude range.

**Compute:** Run locally on CPU or free Colab T4. Encoding 20,000 slices takes approximately 30-40 minutes on T4. Do not use paid RunPod for this step.

| PHASE 3 | Healthy Manifold Learning via Diffusion UNet |
|---|---|

**Model:** `diffusers.UNet2DModel` — trained from scratch. **Scheduler:** `diffusers.DDPMScheduler`

A UNet2DModel with attention blocks at the three deepest resolutions is trained exclusively on the ~20,000 healthy latent tensors. The network never sees a tumour latent during training. The training objective (DDPM MSE loss) is:

$$\mathcal{L} = \mathbb{E}_{z_0,\ \varepsilon \sim \mathcal{N}(0,I),\ t}\left[\|\varepsilon - \varepsilon_\theta(z_t,\ t)\|_2^2\right]$$

*Equation 2. DDPM denoising score-matching objective.*

## Architecture:

| Parameter | Value |
|---|---|
| Sample size (spatial) | 32×32 |
| Input / output channels | 4 (VAE latent channels) |
| Block output channels | (128, 256, 512, 512) |
| Layers per block | 2 |
| Down blocks | DownBlock2D → AttnDownBlock2D × 3 |
| Up blocks | AttnUpBlock2D × 3 → UpBlock2D |
| Attention head dim | 8 |
| Total parameters | ~87 M |

## Training Configuration:

**Optimiser:** AdamW, lr=1e-4, weight_decay=1e-4

**LR Schedule:** Cosine annealing, T_max=3000 epochs, η_min=1e-6

**Mixed precision:** fp16 (torch.cuda.amp) — mandatory on RTX 4090

**Gradient clipping:** max_norm=1.0 to prevent loss spikes

**EMA decay:** 0.9999 — EMA weights used for all inference, not raw checkpoint

**Noise timesteps:** T=1000, linear β schedule: β_start=0.0001, β_end=0.02

**Batch size:** 32 (direct, no gradient accumulation needed with fp16)

**Target epochs:** 3,000 (stop early if loss plateaus below 0.01)

**Checkpointing:** Every 100 epochs + latest.pt for crash recovery

**Compute:** RunPod RTX 4090 — estimated 4–6 hours total

**Logging:** Weights & Biases (free tier) — loss curve + sample reconstructions

## PHASE 4 | Anomaly Extraction via Partial DDIM Inversion

**Scheduler:** `diffusers.DDIMScheduler` (50 deterministic steps). **Model:** EMA weights from Phase 3.

DDIM is not an alternative to DDPM — it is a deterministic inference algorithm that runs on the same trained UNet weights. The stochastic DDPM reverse process is replaced with a deterministic trajectory, ensuring the same test image always produces the same reconstruction. This is non-negotiable for anomaly detection: stochastic inference introduces reconstruction variance that corrupts the residual map with noise that does not originate from the anomaly.

### Inference Algorithm (AutoDDPM — Bercea et al. 2023):

**Step 1 — Encode:** Encode the anomalous test slice through the frozen VAE: z_test = E(x_test) · 0.18215

**Step 2 — Partial Forward:** Apply DDPM closed-form noise injection up to intermediate timestep T_int: z_{T_int} = √(■_{T_int}) · z_test + √(1 – ■_{T_int}) · ε, ε ~ N(0, I)

**Step 3 — DDIM Reverse:** Denoise from T_int back to t=0 using 50 deterministic DDIM steps. The UNet's conditional transitions are bounded to P(z_healthy), so tumour geometry is replaced with plausible healthy tissue.

**Step 4 — Decode:** Decode the reconstructed latent: x_healthy = D(z_0 / 0.18215)

**Step 5 — Residual:** Compute pixel-wise anomaly map: M(x) = |x_test – x_healthy|

$$M(x) = \left|x_{\text{test}} - \hat{x}_{\text{healthy}}\right|$$

*Equation 3. Pixel-wise anomaly localisation map.*

### T_int Hyperparameter Search:

The optimal intermediate timestep T_int is not universally 500. A sweep over T_int ∈ {200, 300, 400, 500, 600} is performed on a 100-slice validation subset. For each value, pixel-level AUROC is computed against BraTS ground truth. The T_int yielding maximum AUROC is used for the full 2,000-slice test set evaluation. The sweep takes approximately 30 minutes and costs under €0.50 on RunPod.

## PHASE 5 | Quantitative Evaluation and Explainability

### Primary Metrics:

AUROC (Area Under the ROC Curve) is the primary metric for unsupervised anomaly detection because it is threshold-free. DICE Similarity Coefficient is computed at the optimal threshold (swept from 0.1 to 0.9 in 0.05 steps). Realistic expectations for this architecture: AUROC 0.72–0.82, DICE 0.35–0.55. These are consistent with published LDM-based unsupervised results on BraTS.

### Explainability via Spatial Self-Attention Maps:

The UNet2DModel's AttnDownBlock2D and AttnUpBlock2D layers compute spatial self-attention at 16×16 and 8×8 resolutions. During inference, PyTorch forward hooks are registered on these layers to capture the attention weight matrices. The maps are upsampled bilinearly to 256×256 and overlaid on the original slice for visual comparison with the residual map M(x). This constitutes interpretability visualisation — it shows spatial consistency between network attention and anomaly localisation, not a mathematical proof of out-of-distribution detection.

```
hook = unet.down_blocks[2].attentions[0].register_forward_hook(capture_attention)
```

## 3. 20-Day Execution Timeline

| Days | Task | Platform | Est. Cost |
|---|---|---|---|
| 1 | Submit BraTS 2021 Synapse registration (time-gated — do this first) | — | Free |
| 1–2 | Environment setup, VAE verification smoke test | Local | Free |
| 3–5 | Phase 1: NIfTI parsing, slice extraction, normalisation (01_preprocess.py) | Local CPU | Free |
| 6 | Phase 2: VAE latent encoding — all healthy slices (02_encode_latents.py) | Colab T4 | Free |
| 7 | Smoke test: 200 latents × 10 epochs on Colab T4 (03_train.py --smoke_test) | Colab T4 | Free |
| 8–12 | Phase 3: Full diffusion UNet training — 3000 epochs (03_train.py) | RunPod 4090 | ~€4 |
| 13–14 | Phase 4: T_int sweep (5 values × 100 slices), full inference on 2k slices | RunPod 4090 | ~€2 |
| 15–16 | Phase 5: AUROC/DICE evaluation, attention map extraction (05_evaluate.py) | Local | Free |
| 17–18 | Result visualisation, anomaly map figures | Local | Free |
| 19–20 | Buffer: second training run if AUROC < 0.70, write-up, defence prep | RunPod 4090 | ~€4 (if needed) |

| Budget Summary | | |
|---|---|---|
| Item | Platform | Estimated Cost |
| Phase 2 — VAE encoding | Colab T4 (free) | €0 |
| Phase 3 — Full training run | RunPod RTX 4090 | ~€4 |
| Phase 4 — Inference + T_int sweep | RunPod RTX 4090 | ~€2 |
| Buffer — second run if needed | RunPod RTX 4090 | ~€4 |
| Debugging buffer | RunPod RTX 4090 | ~€15 |
| **Total planned spend** | | **~€25 of €50 budget** |

## 4. Key Design Decisions and Justifications

### Why T1ce + T2 + FLAIR (not T1)?

T1 provides structural anatomy but poor tumour-to-background contrast. T1ce captures the blood-brain barrier breakdown signature of high-grade glioma. T2 and FLAIR together delineate oedema and infiltration zones. Using these three channels aligns the VAE input with clinical diagnostic protocol and gives the anomaly map clinical interpretability.

### Why sd-vae-ft-mse over training a custom VAE?

Training a KL-VAE from scratch requires an additional 5-7 GPU-days and introduces a second source of hyperparameter sensitivity. The SD VAE encodes 256×256 to exactly 32×32×4 — the target dimensionality — and has been validated on perceptual quality across millions of images. The ft-mse variant is specifically tuned for pixel-level reconstruction fidelity, which is required since the anomaly map is a pixel-wise residual.

### Why DDPM for training but DDIM for inference?

DDPM's training objective (noise prediction via MSE) is well-understood and stable. DDIM's deterministic reverse process does not change training — it reuses the trained UNet with a reformulated denoising equation. Using DDPM for inference introduces stochastic variance in the reconstruction that masks the anomaly signal in the residual map. DDIM removes this variance entirely.

### Why use EMA weights for inference?

Raw training checkpoint weights oscillate around the loss minimum due to stochastic gradient updates. EMA weights are a running average with decay=0.9999, smoothing these oscillations. In diffusion model literature, EMA inference consistently produces better sample quality (lower FID, sharper reconstructions) than the raw checkpoint.

### Why train only on healthy slices?

This is the core unsupervised anomaly detection assumption: a model trained only on normal data cannot reconstruct abnormal regions accurately. When a tumour slice is partially noised and reconstructed, the model inpaints healthy tissue in the tumour region because that is the only distribution it has learned. The residual between original and reconstruction then highlights the anomalous region.

## 5. Literature References

[1] Pinaya et al. (2022). Brain Imaging Generation with Latent Diffusion Models. MICCAI Workshop on Deep Generative Models. Establishes the use of KL-VAE + LDM for medical imaging and justifies latent-space computation over pixel-space.

[2] Wolleb et al. (2022). Diffusion Models for Medical Anomaly Detection. MICCAI 2022. Proves that a diffusion model trained on healthy data learns the normative distribution P_healthy and cannot reconstruct anomalous geometry.

[3] Bercea et al. (2023). Mask, Stitch, and Re-Sample: Enhancing Robustness and Adaptability in Anomaly Detection. AutoDDPM. Derives the partial noising framework and proves the existence of an optimal T_int that preserves global anatomy while destroying high-frequency anomaly features.

[4] Ho et al. (2020). Denoising Diffusion Probabilistic Models. NeurIPS 2020. Original DDPM paper defining the noise schedule and MSE training objective.

[5] Song et al. (2020). Denoising Diffusion Implicit Models. ICLR 2021. DDIM paper. Proves that the DDPM-trained score function admits a deterministic reverse process with equivalent generation quality at 50× fewer steps.